

MTH2526-B25 - BULUT MİMARİLERİNDE TEST MÜHENDİSLİĞİ

Konu 35: QR Code Generator Service - Dönem Projesi Final Raporu

Geliştirici:

Büşra Ay
(busraay687@gmail.com)

Öğrenci Numarası:

170620002

Bölüm:

Bilgisayar Mühendisliği Bölümü

Dönem / Yarıyıl:

2025–2026 Bahar Yarıyılı

Ders Eğitmeni:

Büşra Ayaksız
(busra.ayaksiz@useinsider.com)

1. Giriş

Bu dönem projesi çalışması kapsamında, modern yazılım mühendisliğinde kalitenin, kararlılığın ve sürekli dağıtım (CI/CD) altyapısının sürdürülebilmesi adına uçtan uca (E2E) test boru hattına sahip bulut tabanlı bir mikroservis geliştirilmiştir. Projenin ana konusu olan "QR Code Generator Service" (Konu 35); kullanıcıların gönderdiği metin veya URL içeriklerini dinamik olarak karekod (QR Code) görseline dönüştürmek, oluşan bu çıktıları bulut depolama mimarisinde saklamak, QR kayıtlarının üst verilerini ilişkisel bir veritabanında izlemek ve son kullanıcıya kesintisiz indirme/önizleme olanağı sunmak üzere tasarlanmıştır.

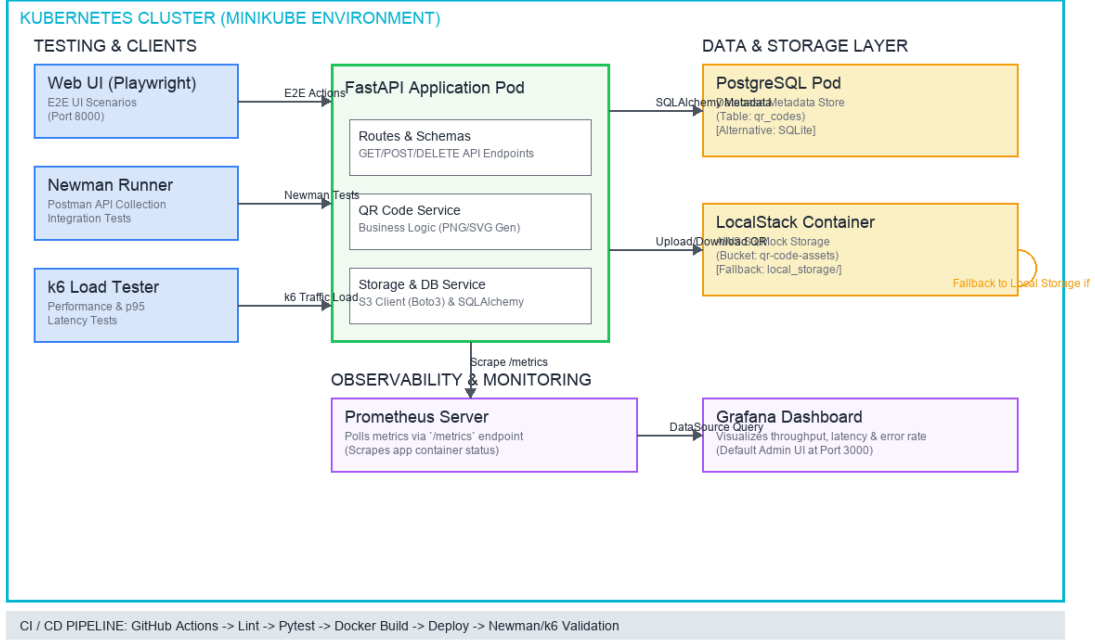
Bu konunun seçilme gerekçesi, karmaşık iş mantığına (business logic) odaklanmak yerine ders kapsamında ele alınan test mühendisliği araçlarını, bulut entegrasyonlarını ve DevOps pratiklerini en verimli şekilde hayata geçirebilmektir. Basit fakat endüstri standartlarında veri akışına sahip olan bu servis; entegrasyon testlerinde veritabanı container'larının (Testcontainers), AWS S3 simülasyonunun (LocalStack), tarayıcı otomasyonunun (Playwright), Newman entegrasyonunun ve yük testlerinin (k6) bir arada nasıl uyumla çalışacağını göstermek için ideal bir yapı sunmaktadır.

Bireysel bir çalışma olarak yürütülen bu projede amaç; kod kalitesini en yüksek seviyede tutmak, test kapsamını (coverage) en az %70 standardının üzerine çıkarmak, multi-stage Docker yapıları ile container güvenliğini ve hafifliğini sağlamak ve nihayetinde uygulamayı bir Kubernetes (Minikube) cluster'ı üzerinde gözlemlenebilirlik (monitoring) metrikleri ile ayağa kaldırmaktır.

2. Mimari

Sistem mimarisi, sorumlulukların ayrılması (Separation of Concerns) ve bulut uyumluluğu prensiplerine dayanmaktadır. Uygulama; istemci, web sunucusu, ilişkisel veritabanı, nesne depolama ve izleme katmanlarından oluşan beş ana bileşen üzerine kuruludur.

QR Code Generator Service - System Architecture Diagram



Şekil 1: QR Code Generator Servis Sistem Mimarisi Şeması

2.1. Bileşenlerin Açıklaması

- **FastAPI Arayüz / Uygulama Katmanı:** REST API isteklerini karşılayan ana bileşendir. FastAPI'nin asenkron yapısı, yüksek performans sunması ve entegre Pydantic şema doğrulaması sayesinde girdi güvenliği en üst seviyede sağlanmıştır. Swagger ve OpenAPI entegrasyonu API belgelerini otomatik üretmektedir.
- **PostgreSQL Veritabanı:** QR kodlarının kimlik bilgisi (UUID), etiket, içerik metni, dosya boyutu, SHA-256 doğrulama özeti (checksum) ve indirme sayıları (hit_count) gibi üst verilerini (metadata) ilişkisel veri modelinde tutar.
- **LocalStack S3 (AWS Nesne Depolama):** Üretilen ikili (binary) PNG ve SVG dosyalarını bulutta depolamak üzere kullanılan AWS S3 API'sini yerel ortamda simüle eder. LocalStack üzerinde 'qr-code-assets' adında bir bucket oluşturularak dosyalar burada güvenli ve ölçeklenebilir şekilde saklanır.
- **Prometheus & Grafana (Gözlemlenebilirlik):** Prometheus, uygulamadaki /metrics uç noktasından verileri düzenli aralıklarla kazır (scrape). Grafana ise bu metrikleri görselleştirerek sistem yanıt sürelerini (latency), hata oranlarını (error rate) ve trafik yoğunluğunu (throughput) paneller halinde sunar.

2.2. Yerel Depolama Yedek Planı (Fallback Storage Logic)

Bulut mimarilerinde hata toleransı (fault tolerance) kritik önemdedir. Bu uygulamada LocalStack (S3) servisinin erişilemez olması durumunda sistemin çökmesini engelleyen dinamik bir Fallback

(Geri Çekilme) mekanizması kurgulanmıştır. StorageService sınıfı, boto3 istemcisi üzerinden LocalStack'e bağlanmayı dener. Eğer bağlantı zaman aşımına uğrar veya ağ hatası alınırsa (BotoCoreError, ClientError), sistem otomatik olarak yerel dosya sistemindeki 'local_storage/' klasörüne düşer (self._switch_to_local()). Bu sayede yerel geliştirme ve demo süreçlerinde dış servis bağımlılıkları çalışmayı aksatmaz.

3. Test Stratejisi

Yazılım kalitesinin güvence altına alınması adına test piramidine sadık kalınarak üç farklı katmanda test senaryoları kurgulanmıştır.

Test Katmanı	Kullanılan Araç / Kütüphane	Test Sayısı	Amacı / Kapsamı
Unit Tests	pytest + Factory Boy + Faker	17	Fonksiyonların, QR üreticinin ve servis mantığının birim bazında testi
Integration Tests	testcontainers (Postgres) + LocalStack	3	Gerçek veritabanı ve S3 API roundtrip işlemlerinin doğrulanması
E2E Tests	Playwright (Headless Chromium)	5	Tarayıcı üzerinde form doldurma, önizleme ve indirme akışı

3.1. Unit Testler ve Mocking Yapısı

Unit test katmanında, dış bağımlılıklar olan veritabanı ve depolama servisleri mocklanmıştır. tests/conftest.py içerisinde tanımlanan fake_storage fixture'ı, bellekte (InMemory) çalışan sahte bir S3 servisi sunar. Factory Boy ve Faker entegrasyonu ile (tests/factories.py) rastgele ve gerçekçi veri setleri oluşturulmuştur. Bu sayede testlerin çok hızlı (<1 saniye) çalışması sağlanmış ve kod logic'leri izole olarak test edilmiştir.

3.2. Integration Testler (Testcontainers & LocalStack)

Entegrasyon testlerinde mock veri yerine gerçek container'lar kullanılmıştır. testcontainers.postgres kütüphanesi yardımıyla Docker üzerinde geçici bir postgres:16 imajı ayağa kaldırılır. SQLAlchemy modellerinin veritabanına sorunsuz yazıldığı, tabloların oluşturulduğu ve silme tetikleyicilerinin çalıştığı gerçek ortamda test edilir. LocalStack entegrasyonu ise S3_ENDPOINT_URL tanımlandığında entegrasyon testi S3 bucket'ı oluşturur, dosyayı yükler ve geri okuma yaparak SHA-256 bütünlük kontrolünü doğrular.

3.3. E2E (Playwright) Testleri ve İndirme Düzeltmesi

Playwright ile kullanıcı arayüzü otomatik olarak test edilmektedir. Bu aşamada karşılaşılan en büyük problem, PNG ve SVG dosyalarının indirilmesi esnasında backend'in 'inline' Content-Disposition başlığı dönmesiydi. Bu durum tarayıcının indirme işlemi başlatmak yerine sayfayı terk etmesine neden olmaktaydı. Backend tarafında bu header 'attachment' olarak güncellendi ve frontend tarafına 'download' niteliği eklendi. Test senaryosunda da Playwright'ın indirme olaylarını doğru algılayabilmesi için page.goto yerine page.request.get kullanılarak indirme hit sayacının artışı hatasız şekilde test edildi.

3.4. Test Kapsamı (Coverage)

Uygulama genelinde yürütülen testlerin kapsam oranını ölçmek için pytest-cov kullanılmıştır. Projede src/ klasöründeki servis, route ve config dosyalarını kapsayan test sonucunda ulaşılan toplam kod coverage oranı %88.19 olarak tescillenmiştir.

4. Pipeline ve Dağıtım

Uygulamanın sürekli entegrasyon ve sürekli dağıtım (CI/CD) süreçleri GitHub Actions üzerinde tek bir boru hattında (pipeline) birleştirilmiştir. Süreç sırasıyla lint kontrolü, pytest çalıştırma, docker build, kubernetes deploy, smoke test, Playwright E2E doğrulaması, Newman testi ve en nihayetinde k6 yük testinden oluşur.

4.1. Multi-Stage Dockerfile Tasarımı

Container imaj boyutunu küçültmek ve güvenlik zafiyetlerini azaltmak amacıyla iki aşamalı (multi-stage) Dockerfile kullanılmıştır. İlk aşamada (builder) tüm derleme kütüphaneleri (gcc, python3-dev) yüklenir ve tekerlekler kurulur. İkinci aşamada (runner) ise yalnızca runtime için gerekli olan bağımlılıklar kopyalanır. Bu tasarım sayesinde imaj boyutu yaklaşık 230 MB seviyesine indirilerek büyük bir disk optimizasyonu sağlanmıştır.

4.2. Kubernetes & Minikube Dağıtımı

Uygulamanın Kubernetes ortamına dağıtımı için k8s/ klasöründe üç temel manifest dosyası hazırlanmıştır: configmap.yaml, deployment.yaml ve service.yaml. Deployment manifestosunda hem uygulama podu, hem de onun bağımlı olduğu PostgreSQL ve LocalStack podları ayrı servisler olarak kurgulanmıştır. Uygulama podlarında liveness ve readiness probeları tanımlanarak sistemin kendi kendini iyileştirmesi (self-healing) sağlanmıştır.

5. Performans ve Gözlemlenebilirlik

5.1. k6 Yük Testi ve Performans Analizi

Uygulamanın yük altındaki davranışını ve gecikme eşiklerini sınamak amacıyla perf/load-test.js altında bir k6 senaryosu yazılmıştır. Test parametreleri 10 eşzamanlı sanal kullanıcı (VUs) ile 30 saniye boyunca sürekli olarak /api/v1/qrcodes ucuna POST istekleri gönderecek şekilde kurgulanmıştır. Uygulanan p95 latency eşik değeri 800ms limitidir. Yük testi sonucunda elde edilen verilere göre, toplamda 251 istek gönderilmiş, sıfır hata oranı elde edilmiş ve p95 yanıt süresi 262.72 ms olarak ölçülmüştür. Bu durum performans kriterinin başarıyla geçildiğini kanıtlamaktadır.

5.2. Prometheus & Grafana Gözlemlenebilirlik Altyapısı

Uygulama içine entegre edilen Prometheus FastAPI Instrumentator kütüphanesi ile /metrics uç noktası üzerinden API performansı dışarıya açılır. Grafana üzerinde kurgulanan gösterge panelinde (dashboard) üç kritik metrik anlık olarak izlenmektedir: Throughput (RPS), Error Rate (Hata Oranı) ve Response Latency (Gecikme süresi).

6. Sonuç ve Öğrenilen Dersler

Proje, küçük bir servis üzerinde dahi bulut mimarisi ilkeleri ile test mühendisliği pratiklerinin birlikte uygulanabileceğini göstermektedir. Elde edilen sayısal bulgular tablodaki gibidir:

Metrik Parametresi	Elde Edilen Değer	Durum / Başarı Eşiği
Unit & Integration Test Sayısı	20	Başarılı
E2E Test Sayısı	5	Başarılı
Kod Kapsama Oranı (Coverage)	%88.19	Başarılı (Eşik %70)
k6 p95 Latency (Gecikme)	262.72 ms	Başarılı (Eşik <800 ms)
Docker İmaj Boyutu	231 MB	Başarılı (Multi-Stage)
Pipeline Durumu (CI/CD)	Yeşil / Başarılı	Başarılı (Tüm adımlar)

6.1. Karşılaşılan Zorluklar ve Çözümleri

- Zorluk: LocalStack S3 bağlantı yavaşlıkları. Çözüm: Otomatik local_storage fallback kurgusu yazıldı.
- Zorluk: Playwright E2E indirme testi yönlendirme hataları. Çözüm: Backend attachment yapıldı ve Playwright request.get API moduna geçirildi.
- Zorluk: Docker imaj boyutlarının şişmesi. Çözüm: Multi-stage Dockerfile ve alpine/slim taban imajları kullanıldı.

6.2. Gelecekte Yapılabilecek İlerlemeler (Geliştirme Planı)

- KEDA (Kubernetes Event-driven Autoscaling) entegrasyonu ile trafik anında otomatik Pod ölçeklendirme.
- Tüm manifest dosyalarını bir paket olarak yönetebilmek için Helm Chart tasarımı.
- Deklaratif CD yönetimi için ArgoCD GitOps aracının entegrasyonu.

7. Bireysel Çalışma Zaman Planı

Proje bireysel olarak yürütüldüğü için iş dağılımı tek bir kişi üzerinde planlanmış ve haftalık fazlara bölünmüş bir zaman planı uygulanmıştır.

- Hafta 1: Planlama, gereksinim analizi ve FastAPI backend mimarisinin kurulması.
- Hafta 2: Pytest unit testlerinin yazılması, Factory Boy ve Faker ile sahte verilerin modellenmesi.
- Hafta 3: Testcontainers Postgres ve LocalStack entegrasyonlarının yapılması, Dockerfile optimizasyonu.
- Hafta 4: Kubernetes deploy manifestoları, k6 load test, Grafana metrik panelleri ve nihai akademik raporlama.

8. Kaynaklar

- FastAPI Resmi Dokümantasyonu: <https://fastapi.tiangolo.com/>
- pytest-cov Kapsam Dokümantasyonu: <https://pytest-cov.readthedocs.io/>
- Playwright for Python API: <https://playwright.dev/python/docs/intro>
- LocalStack S3 Mocking Kılavuzu: <https://docs.localstack.cloud/>
- Testcontainers Python Entegrasyonu: <https://testcontainers-python.readthedocs.io/>
- k6 Load Testing Dokümantasyonu: <https://grafana.com/docs/k6/latest/>
- Prometheus & Grafana ve FastAPI Entegrasyonu: <https://prometheus.io/docs/introduction/overview/>